EcoHealth Risk Monitor

Technical Report

Internet Solutions Engineering

Department of Computer Science, Maynooth University | 2024–2025

Module	Project	Assessment	Stack
Internet Solutions Engineering	EcoHealth Risk Monitor	Technical Report (15% CA)	MERN + Docker + Groq AI

1. Introduction

EcoHealth Risk Monitor is a full-stack MERN web application that addresses a significant public health challenge: the disconnect between raw environmental data and personalised, actionable health guidance. While numerous air quality dashboards exist that surface real-time pollutant readings, they present data in formats that are difficult for everyday users to interpret — particularly those with underlying health conditions such as asthma, chronic respiratory disease, or age-related vulnerabilities.

The core motivation stems from well-established research confirming that exposure to air pollution disproportionately affects vulnerable populations. A raw AQI reading of 120 communicates very different levels of risk to a healthy 25-year-old compared to a 70-year-old with asthma, yet public dashboards treat all users identically. EcoHealth resolves this by combining real-time environmental data from two external APIs (OpenAQ for air quality, OpenWeather for meteorological conditions) with a user-submitted health sensitivity profile, computing a personalised risk score with human-readable recommendations.

The system uses a dual-engine architecture: a deterministic rule-based calculator for reliability and an AI language model (Groq Llama 3.3 70B) for nuanced, context-aware guidance. This combination ensures both predictability and depth of output, and is toggled via an environment variable using the Strategy design pattern.

1.1 Project Objectives

- Deliver personalised environmental health risk scores by combining real-time air quality and weather data with individual user health profiles
- Integrate at least two external data sources (OpenAQ, OpenWeather) with robust error handling, retry logic, and response caching
- Implement a full MERN-stack multi-tier architecture following MVC, Repository, and Strategy design patterns
- Provide configurable location-based alert notifications that automatically evaluate environmental risk against user-defined thresholds
- Support the full user lifecycle — registration, authentication, profile management, and secure password recovery
- Deploy a fully containerised, reproducible system via Docker and docker-compose with a structured GitLab branching workflow
- Apply a hybrid AI/rule-based risk engine capable of producing context-aware, personalised health recommendations

The system supports three user types: guest users who access location-based risk assessments without registration; registered users who receive profile-personalised scores, saved locations, alerts, and historical tracking; and administrators who monitor API health, manage users, and control system configuration.

2. Development Strategy

The project was structured around a phased iterative development model comprising ten sequential backend phases followed by frontend development. This approach was chosen over a Scrum-style sprint model because the team needed a predictable, dependency-aware workflow: the frontend could not be developed until backend endpoints were verified, and the AI integration could not be reliably tested until the rule-based engine was stable.

2.1 Phased Development Plan

Backend Phases:

Phase	Name	Key Deliverable
1	Project Setup	Git repository, MERN scaffold, MongoDB connection, Docker skeleton
2	Authentication	JWT registration/login, bcrypt hashing, route guards, Joi validation
3	Location & APIs	OpenAQ + OpenWeather integration, geocoding, retry logic
4	Risk Engine	Rule-based AQI/weather scorer, sensitivity multipliers, explainability object
5	Dashboard API	GET /api/risk aggregator combining all data sources and risk calculation
6	User Profiles	Health sensitivity settings, saved locations, home location management
7	Alerts System	Alert rules in MongoDB, node-cron scheduler, Nodemailer email delivery
8	Historical Data	RiskHistory model, 30-min throttle, 50-record cap, paginated history
9	Admin Backend	System metrics, user management, API status monitoring, cache control
10	Optimisation	node-cache (10 min TTL), rate limiting, MongoDB compound indexing

Frontend Phases:

Phase	Feature Area	Key Deliverable
F1	Project Setup	Vite + React scaffold, Tailwind CSS, Axios with JWT interceptor, AuthContext
F2	Authentication Pages	Login, Register with optional health profile, Forgot/Reset Password flow
F3	Dashboard	Risk score card, AI reasoning display, AQI/weather cards, Leaflet map click-to-select
F4	Alerts Page	Create alerts with threshold slider, location search confirmation, toggle/delete
F5	History Page	Recharts risk trend chart, summary stats, paginated records
F6	Profile Page	Health data update, saved locations, home location management
F7	Admin Panel	System overview, users table with role management, API status on-demand
F8	Deployment	Nginx config for React Router, multi-stage Dockerfile, docker-compose integration

2.2 Justification of Approach

Each phase had defined goals and API contracts documented before implementation began. Backend endpoints were tested using Postman before any frontend work was initiated. This separation was a deliberate risk-management decision: building the frontend against unverified API contracts would have introduced compounding bugs and rework.

The AI integration was added only after Phase 4 was verified stable, using the Strategy pattern to enable hot-swapping between engines via an environment variable. This ensured the system remained functional at all times during AI integration. Team responsibilities were split by architectural layer — backend, frontend, and database integration — to minimise merge conflicts and allow parallel workstreams once interface contracts were agreed. Version control followed a structured branching strategy: a backend branch and a frontend branch, both merged into main via GitLab merge requests.

3. Implementation

3.1 System Architecture

EcoHealth follows a strict three-tier architecture with clear separation between the presentation layer (React/Nginx), business logic layer (Node.js/Express), and data layer (MongoDB). This supports independent scaling, maintainability, and testability of each tier.

System Architecture Overview

PRESENTATION LAYER (Port 80): React 18 + Vite 5 + Tailwind CSS, served via Nginx Alpine container ↕ REST API (Axios + JWT Bearer Token)BUSINESS LOGIC LAYER (Port 5050): Node.js 20 + Express.js (MVC), Risk Engine (Strategy Pattern), JWT Auth + Joi Validation + node-cache ↕ Mongoose ODMDATA LAYER (Port 27017): MongoDB 7 (persistent named Docker volume) ↕ External APIsEXTERNAL SERVICES: OpenAQ v3 (Air Quality), OpenWeather (Weather), OpenCage (Geocoding), Groq (AI Risk Engine), Gmail/Nodemailer (Email Alerts)

3.2 Backend Structure and Design Patterns

The backend follows the MVC (Model-View-Controller) pattern with Express routers acting as controllers, Mongoose models defining the data layer, and service classes encapsulating business logic. A Repository pattern abstracts database access, ensuring controllers never interact with MongoDB directly. This separation allows individual layers to be tested and replaced independently.

The most significant architectural decision was the application of the Strategy Pattern to the risk engine. Two concrete strategy implementations — RuleBasedStrategy and AIStrategy — share a common interface. The active strategy is selected at runtime via the RISK_ENGINE_MODE environment variable, enabling hot-swapping without any code changes. When the AI strategy is active, if the Groq API fails or times out, the system silently falls back to the rule-based strategy, ensuring 100% availability of risk scoring.

3.3 Risk Scoring Engine

The rule-based engine computes a composite score from AQI, humidity, and wind speed, then applies user sensitivity multipliers. Every risk response includes an explainability object breaking down contributing factors — an architectural decision driven by transparency requirements and an AI assistant suggestion during development.

Condition	Multiplier Effect	Rationale
Asthma	+0.3 on base score	Respiratory sensitivity to particulate matter
Age > 60	+0.2 on base score	Reduced immune and respiratory resilience
Other chronic conditions	+0.1 per condition	Cumulative vulnerability from comorbidities
No profile (guest)	×1.0 (default)	Conservative baseline for unknown health status

The AI engine (Groq Llama 3.3 70B) receives both the environmental data and the user health profile and generates context-sensitive recommendations that outperform the rule-based approach for complex multi-condition profiles. The active engine mode is visible in the Admin Panel under the system metrics card.

3.4 Key API Endpoints

Endpoint	Method	Description	Auth
/api/risk	GET	Core dashboard: fetches AQI, weather, computes risk, returns recommendations. Supports ?query=, ?lat=&lon=, or auto-detect. 30-min throttle on history saving.	Optional
/api/auth/register	POST	User registration with optional health sensitivity profile (age, asthma, conditions)	None
/api/auth/login	POST	JWT authentication, returns signed 7-day token	None
/api/auth/forgot-password	POST	Sends password reset email with 1-hour secure token	None
/api/auth/reset-password	POST	Verifies reset token and updates hashed password	None
/api/users/profile	GET/PUT	Retrieve or update health profile and saved locations	JWT
/api/alerts	POST/GET	Create/retrieve alert configurations with threshold slider	JWT
/api/alerts/:id/toggle	PATCH	Toggle alert active/paused state	JWT
/api/history	GET	Paginated risk history (max 50 per user), trend data	JWT
/api/location	GET	Resolve location name to coordinates + live environmental preview	None
/api/admin/metrics	GET	System uptime, memory, Node version, current risk engine mode	Admin
/api/admin/stats	GET	Aggregated user count, avg risk score, top locations, risk distribution	Admin
/api/admin/api-status	GET	Live ping of OpenAQ and OpenWeather APIs	Admin
/api/admin/refresh	POST	Force cache clear for specific coordinates or full cache flush	Admin
/api/admin/users/:id/role	PATCH	Promote or demote user to admin role	Admin

3.5 Database Schema Design

MongoDB was selected for its flexible document model, which accommodates variable user health profiles without requiring schema migrations. Four primary collections are used:

Collection	Key Fields	Indexes
User	email (unique), password (bcrypt), name, role (user/admin), sensitivityProfile {age, asthma, otherConditions[]}, savedLocations[], resetPasswordToken, resetPasswordExpires	email (unique)

Collection	Key Fields	Indexes
Alert	userId (ref), location {name, lat, lon}, threshold (0–100), emailEnabled, active, lastTriggered, lastChecked	userId, active (compound)
RiskHistory	userId (ref), location, environment {airQuality, weather}, risk {score, level, recommendations[]}, createdAt	userId + createdAt (compound), location.name
(Scheduler)	Runs every 30 min via node-cron — evaluates all active Alert documents against live risk data. Does NOT write to RiskHistory.	N/A

Joi schema validation is applied at the middleware layer before any data reaches Mongoose models, providing a double layer of validation: structural validation (Joi) and type/constraint enforcement (Mongoose). The RiskHistory collection is capped at 50 records per user — when a new record is created that exceeds this limit, the oldest records are automatically deleted.

3.6 History Throttling and 30-Minute Cache Logic

A deliberate design decision was made to prevent excessive history accumulation. When the dashboard loads with no manual search, the system checks for a recent RiskHistory record for the same location within the last 30 minutes. If found, it returns the cached history record directly without calling external APIs. A manual search or map click bypasses this check and always fetches fresh data, saving a new history record. This reduces unnecessary API calls, keeps history meaningful, and prevents the alert scheduler's background evaluations from polluting user history.

3.7 Performance Optimisation

External API calls were the primary performance bottleneck. A node-cache layer with a 10-minute TTL per coordinate pair was implemented, reducing GET /api/risk response time from 649ms to 7ms for cached coordinates — a 99% improvement. Rate limiting (express-rate-limit) was applied at three tiers: 100 requests per 15 minutes generally, 10 for authentication endpoints, and 20 for risk calculations. MongoDB compound indexes on userId+createdAt and active status enable efficient alert evaluation across growing datasets.

3.8 Frontend Architecture Overview

The React frontend is built with Vite 5 and styled with Tailwind CSS. React Router v6 manages three route types: public routes (login, register, forgot/reset password), protected routes requiring a valid JWT, and admin-only routes that additionally check the user role. Axios is configured with a JWT interceptor attaching Bearer tokens to all authenticated requests, and a global 401 handler that clears localStorage and redirects to login on token expiry. Session data including the last searched location is cleared entirely on logout.

3.9 State Management and Data Flow

Authentication state is managed via a React Context (AuthContext) that persists the JWT token and user object in localStorage across sessions. The dashboard uses a combination of component state and localStorage to maintain the last searched location between page refreshes. sessionStorage is intentionally not used — the 30-minute throttle on the backend prevents unnecessary re-fetching more effectively than client-side session caching. All API communication is centralised through a single Axios instance, ensuring consistent authentication headers and error handling across all components.

3.10 Component Architecture

The frontend is structured into pages and reusable components with clear separation of concerns. Pages own data fetching and state; components are presentational only. The Navbar component reads the user role from AuthContext to conditionally render the Admin tab. The LocationMap component encapsulates all Leaflet logic, exposing only an onLocationClick callback to the parent Dashboard page. The Recharts LineChart in the History page uses a custom tooltip component that displays location name, date, and risk level on hover.

3.11 Form Validation Strategy

All forms use React Hook Form with validation rules manually aligned to the backend Joi schemas. This means validation fires client-side before any network request is made, while the backend remains the authoritative source of truth. The register form mirrors the Joi schema for sensitivityProfile, making all health fields optional with appropriate type coercion (age as integer, asthma as boolean, otherConditions as comma-separated string parsed to array before submission).

3.12 Map Integration (UC4/UC5)

Location selection is implemented using Leaflet via React Leaflet. The map auto-centers on the currently loaded location coordinates whenever new risk data loads. A custom click handler captures latitude and longitude from any map click, passes them to fetchRisk with force=true (bypassing the 30-minute cache), and updates the map marker and all dashboard cards simultaneously. The browser Geolocation API (UC5) is integrated as a separate button that requests device location permission and feeds the resulting coordinates through the same fetchRisk path.

3.13 Admin Panel Architecture

The admin panel is implemented as a single-page tabbed interface with four tabs: Overview, Users, Alerts, and API Status. The Overview tab loads metrics and stats on mount. The API Status tab deliberately fetches on demand rather than on mount — this avoids making live external API calls every time an admin opens the panel. The Users tab renders a live role-change select dropdown that calls PATCH /api/admin/users/:id/role immediately on change, with the updated user reflected in the table without a page reload. The Clear Cache button calls POST /api/admin/refresh with an empty body, flushing the entire node-cache.

Page / Component	Key Technology	Features
Login	React Hook Form	Email/password validation, show password toggle, success message after reset
Register	React Hook Form + Joi-aligned	Optional health profile (age, asthma, conditions) at registration
Forgot / Reset Password	React Hook Form + URL token	Secure email reset flow, 1-hour expiry, token validation on reset page
Dashboard	Recharts, Leaflet, Axios	Risk score ring, AI reasoning, AQI/weather cards, Leaflet map click-to-select, 30-min throttle
Alerts	Range slider, Axios	Create with threshold slider (10–90), location search confirmation, toggle/delete

Page / Component	Key Technology	Features
History	Recharts LineChart	Risk score trend, summary stats, paginated records, 50-record cap
Profile	React Hook Form	Health data update, saved locations with home designation, Set as Home
Admin	Tab navigation, Axios	Overview stats, users table with role change, alerts view, API status on-demand, Clear Cache
Navbar	React Router useLocation	Active link highlighting, admin tab conditional, mobile hamburger menu, user role display

4. Vibe Coding Strategy

The entire EcoHealth project was developed using Claude AI (claude.ai) as the primary development assistant, representing a deliberate and structured vibe coding approach. The interaction spanned approximately 150 or more back-and-forth exchanges across all ten backend phases, the complete frontend, debugging sessions, deployment configuration, and this report — covering the full development lifecycle from architecture to submission.

4.1 Methodology

The approach followed a consistent pattern: each feature was explained conceptually before requesting implementation, errors were debugged in real time by sharing terminal output and stack traces directly in conversation, and each proposed fix was verified by running the system before proceeding. This verification-first discipline was critical — the most productive sessions were those where a single change was tested before the next was requested.

The AI was used not merely as a code generator but as an architectural collaborator. Several of the most impactful design decisions emerged from AI suggestions during development conversations rather than from the initial specification.

4.2 Concrete Examples of AI Impact

AI Contribution	Impact on System
Strategy Pattern for Risk Engine	Suggested hot-swap interface for AI/rule-based engines via env variable — adopted as a core architectural pattern across both engines
Service Naming Bug (locationService vs geocodingService)	Identified a startup crash caused by a module naming mismatch that was non-obvious from reading individual files in isolation
Explainability Object	Proposed adding a factor breakdown to every risk response — makes scoring transparent and directly supports the examiner's ability to trace the calculation
node-cache over Redis	Recommended a simpler caching approach appropriate to single-server deployment — avoided unnecessary infrastructure complexity for the project scope
Optional Auth Middleware	Designed middleware allowing /api/risk to serve both guest (IP-based) and authenticated (profile-based) users from the same endpoint
sensitivityProfile Save Bug	Caught that the health profile was not being saved during registration because the parameter was not being passed through the auth service layer
30-Minute History Throttle	Proposed database-level history deduplication check to prevent the alert scheduler from polluting user history with background evaluations
50-Record History Cap	Suggested auto-pruning oldest records on insert to keep history meaningful without requiring manual cleanup or pagination issues
Password Reset Flow	Designed the full forgot/reset password feature including crypto token generation, 1-hour expiry, and secure email delivery

AI Contribution	Impact on System
Docker Networking	Resolved MONGO_URI configuration for Docker internal networking (mongodb://mongodb:27017) vs local development connection strings

4.3 Critical Reflection

The most significant learning from this vibe coding experience is that the value of AI assistance is maximised through iterative verification rather than bulk generation. Early in the project, accepting larger blocks of AI-generated code without immediate testing led to errors that were difficult to trace. Shifting to a one-step-at-a-time, test-before-proceeding discipline dramatically improved both development velocity and code quality.

The AI demonstrated consistent strengths in architectural reasoning and pattern identification. It could identify structural problems that were not immediately obvious from reading individual files. Its weaknesses were in environment-specific details — Docker networking nuances, Vite version compatibility, and API-specific error codes — where testing was always required to verify suggestions. The AI also occasionally suggested variable names or method signatures that did not match the existing codebase, necessitating careful comparison rather than direct adoption.

Overall, the AI coding assistant functioned most effectively as a senior developer with broad architectural knowledge but limited visibility into the specific runtime environment. The human developer's role was to provide environmental grounding, verify outputs, and make final architectural judgements. Neither role was sufficient alone.

5. Deployment

5.1 Container Architecture

EcoHealth is deployed as three Docker containers orchestrated via a single docker-compose.yml file, ensuring a fully reproducible environment launchable with one command from any machine with Docker installed.

Container	External Port	Configuration
ecohealth_frontend	80	Multi-stage React build; production bundle served via Nginx Alpine; React Router handled by try_files fallback
ecohealth_backend	5050	Node.js 20 Alpine; Express server; connects to MongoDB via internal Docker network
ecohealth_mongo	None (internal)	MongoDB 7 official image; persistent named volume (mongo_data) for data survival across restarts

5.2 GitLab Workflow and Branching Strategy

The project uses a structured GitLab branching strategy aligned with team responsibilities. The backend developer committed work phase-by-phase to a dedicated backend branch, with each commit corresponding to a specific development phase. The frontend team cloned the repository, created a frontend branch, and committed their React components by feature area. Both branches were merged into main via GitLab merge requests, providing a clear audit trail of contributions.

Branch	Owner	Content
main	Team Lead	Final merged codebase, docker-compose.yml, root .gitignore, project README
backend	Backend Developer	10 phased commits covering all Express/Node.js/MongoDB work, Dockerfile, .env.example
frontend	Frontend Team	Phased commits covering React pages, components, Tailwind setup, Nginx config, Dockerfile

5.3 Deployment Process

Only the frontend port (80) is exposed externally. Backend and MongoDB are accessible only within the Docker internal network, improving security posture. Environment variables are injected via a root .env file, with an .env.example template provided for reproducibility. A MongoDB health check in docker-compose.yml ensures the backend waits for the database before starting. The complete deployment sequence is:

> **One-Command Deployment**
> git clone <repository-url>cp .env.example .env # populate API keysdocker-compose up --buildFrontend: http://localhost (port 80)Backend API: http://localhost:5050

5.4 Deployment Challenges and Resolutions

Challenge	Resolution
Vite version incompatibility with Node.js 20.14	Downgraded to Vite 5 and @vitejs/plugin-react 4, which restored build stability in the Docker container
Gemini API free tier had zero quota	Switched to Groq (Llama 3.3 70B) — fully free tier with generous rate limits, better response quality
IP geolocation fails on localhost	Explicit localhost IP detection with Dublin, Ireland substituted as a development fallback
Docker MongoDB starts as fresh database	Documented in README — requires seed script or manual data entry; existing local data does not transfer automatically
FRONTEND_URL undefined in reset email	Added FRONTEND_URL environment variable to both backend .env and docker-compose.yml service configuration
React Router 404 on Nginx direct URL access	Configured nginx.conf with try_files $uri $uri/ /index.html to handle client-side routing correctly

6. Conclusions and Future Work

6.1 Critical Evaluation

EcoHealth successfully achieves its core objective: delivering a personalised environmental health risk assessment that meaningfully differs from existing raw AQI dashboards. The system is fully functional across all intended use cases — from guest location-based risk lookups to authenticated user profiles with alert notifications, password reset flows, and historical trend tracking with intelligent throttling.

The phased development strategy proved to be the single most effective process decision. By enforcing backend verification before frontend development and AI integration after rule-based stability, the project avoided cascading failures. The Strategy Pattern for the risk engine was particularly valuable: toggling between AI and rule-based engines via an environment variable reduced risk during AI integration and enabled rapid comparison of output quality.

Caching was the most impactful technical optimisation — reducing dashboard response times by 99% (649ms to 7ms). The 30-minute history throttle and 50-record cap were later additions that significantly improved data quality and system behaviour, demonstrating that iterative refinement during development continues to yield value even after core features are complete.

6.2 What Did Not Go as Planned

The most significant deviation from the original specification was the planned use of SendGrid for email notifications. The final implementation uses Nodemailer with Gmail SMTP, which provided equivalent functionality at zero cost. This was a pragmatic decision justified by simplicity and cost, and has been clearly documented as a specification deviation.

The Gemini API free tier had zero practical quota, requiring a mid-project switch to Groq. While resolved without significant delay, it highlights the importance of testing external API availability — not just key validity — during planning. Admin alert template management described in the specification was not implemented; email templates remain hardcoded. This is a known gap between specification and delivery. Structured logging was also absent — the system uses console.log rather than a logging library, which would make production incident diagnosis significantly more difficult.

6.3 Future Work

The most impactful future direction is transitioning from the current Groq-based AI to a purpose-trained machine learning model. As the system accumulates risk scores and health outcomes, a supervised learning approach could produce substantially better personalised predictions than a general-purpose LLM. Immediate priorities include:

- CI/CD pipeline via GitHub Actions deploying to Docker Hub with automatic container updates on merge to main
- Redis replacing node-cache for distributed caching when scaling to multiple server instances
- WebSocket implementation for real-time risk updates without page refresh
- Progressive Web App manifest and service worker for mobile installation and offline support
- Push notifications via Web Push API to supplement the current email alert system
- CSV and PDF export of risk history data for sharing with healthcare providers
- Global default threshold management for administrators, replacing hardcoded defaults

- Structured logging via Winston or Pino for production-grade incident diagnosis
- User feedback endpoint (POST /api/history/:id/feedback) to begin collecting labelled training data for future ML model

In summary, EcoHealth demonstrates that a well-structured iterative approach, thoughtful design pattern selection, and disciplined use of AI assistance can produce a production-quality full-stack MERN application within an academic project timeline. The gaps identified are honest and addressable rather than fundamental, and the system provides a credible foundation for the future development directions described.

Appendix A: Technology Stack Summary

Layer / Concern	Technology	Version / Notes
Frontend Framework	React + Vite	React 18, Vite 5
Styling	Tailwind CSS	v3 — utility-first, no compiled CSS needed
Routing	React Router	v6 — protected, admin, and public route types
HTTP Client	Axios	JWT interceptor + global 401 handler
Forms	React Hook Form	Client-side validation aligned to backend Joi schemas
Charts	Recharts	LineChart for risk trend, custom tooltip
Maps	Leaflet + React Leaflet	Click-to-select location, auto-centered on data load
Backend Runtime	Node.js	v20 Alpine (Docker)
Backend Framework	Express.js	MVC pattern, middleware chain
Database	MongoDB + Mongoose	v7 official image, named persistent volume
Authentication	JWT + bcrypt	7-day tokens, bcrypt password hashing
Validation	Joi	All request bodies validated before controller
AI Engine	Groq Llama 3.3 70B	Free tier, falls back to rule-based if unavailable
Email Notifications	Nodemailer + Gmail	Alert emails + password reset flow
Task Scheduler	node-cron	Every 30 minutes — alert evaluation
Caching	node-cache	10 min TTL per coordinate pair
Rate Limiting	express-rate-limit	100/15min general, 10/15min auth, 20/5min risk
Deployment	Docker + Nginx + docker-compose	Three-container orchestration
Version Control	GitLab	backend + frontend branches, merged to main
External APIs	OpenAQ v3, OpenWeather, OpenCage	Air quality, weather, geocoding

Appendix B: Use Case Implementation Coverage

B.1 Main System Use Cases

UC	Description	Status	Implementation
UC1	Register Account	✅ Complete	POST /api/auth/register with optional sensitivityProfile
UC2	Login	✅ Complete	POST /api/auth/login — returns JWT token
UC3	Logout	✅ Complete	Client-side token removal from localStorage on logout
UC4	Search Location	✅ Complete	Dashboard search bar → GET /api/location → GET /api/risk?query=
UC5	Detect Current Location	✅ Complete	Browser Geolocation API → coordinates → GET /api/risk?lat=&lon=
UC6	Retrieve Environmental Data	✅ Complete	environmentService calls OpenAQ + OpenWeather in parallel
UC7	Handle API Failure	✅ Complete	retryHandler.js + AI→rule-based fallback + IP→Dublin fallback
UC8	View Dashboard	✅ Complete	GET /api/risk aggregates all data — frontend renders risk card, map, cards
UC9	Manage User Profile	✅ Complete	GET/PUT /api/users/profile — health data + saved locations
UC 10	Calculate Personalised Risk Score	✅ Complete	riskEngine.js — rule-based or Groq AI with sensitivity multipliers
UC 11	Generate Recommendations	✅ Complete	Both engines produce recommendations array; AI adds reasoning field
UC 12	Configure Alerts	✅ Complete	POST /api/alerts — location + threshold slider + email toggle
UC 13	Trigger Alert	✅ Complete	node-cron scheduler runs every 30 minutes — evaluates all active alerts
UC 14	Send Notification	✅ Complete	Nodemailer + Gmail SMTP — personalised HTML email with risk details
UC 15	View Historical Trends	✅ Complete	GET /api/history — Recharts LineChart trend + paginated records
UC 16	Save Location	✅ Complete	PUT /api/users/profile savedLocations[] — home location = index 0
—	Forgot/Reset Password	✅ Added	POST /api/auth/forgot-password + /reset-password — crypto token flow

B.2 Admin System Use Cases

UC	Description	Status	Notes
Admin UC1	Admin Login	✅ Complete	Role-based JWT — admin role required for all /api/admin/* routes

UC	Description	Status	Notes
Admin UC2	View System Health Dashboard	✅ Complete	GET /api/admin/metrics — uptime, memory, Node version, risk engine mode
Admin UC3	Monitor API Status / Rate Limits	✅ Complete	GET /api/admin/api-status — live ping of OpenAQ and OpenWeather
Admin UC4	View Logs and Errors	⚠️ Partial	Console logging only — no persistent log storage implemented
Admin UC5	Manage Threshold Rules	⚠️ Partial	Per-user thresholds exist; no global default threshold management
Admin UC6	Manage Alert Templates	⚠️ Not implemented	Email template hardcoded in emailService.js — future work item
Admin UC7	Manage Users	✅ Complete	GET/DELETE /api/admin/users, PATCH role, GET user history
Admin UC8	Force Data Refresh	✅ Complete	POST /api/admin/refresh — clears node-cache for specific coords or all

References

Arvind, D.K. and Maiya, S. (2023) Personalised environmental health risk systems. [See specification document for full citation.]

World Health Organization (2021) WHO global air quality guidelines: particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide. Geneva: WHO Press.

Cheng, S. et al. (2024) Variable importance measure for spatial machine learning models with application to air pollution exposure prediction. Environmental Health Perspectives.

Institute for Health Metrics and Evaluation (2020) Global burden of disease attributable to ambient air pollution. Seattle: IHME.

Zounemat-Kermani, N. et al. (2025) An AI-driven framework for the prediction of personalised health response to air pollution. Nature Digital Medicine.